

Bases de Données: Introduction à l'optimisation

Chapitre 02: Dispositifs de Stockage

Asma BERRIRI

asma.berriri@universite-paris-saclay.fr

Page web du cours :

[https://sites.google.com/view/asmaberriri/home/
aspects-systeme-base-de-donnees](https://sites.google.com/view/asmaberriri/home/aspects-systeme-base-de-donnees)

Table of Contents

- 1 Supports de stockage
- 2 Gestion des mémoires
- 3 Enregistrements, blocs et fichiers

Techniques de stockage :

- Où sont les données ? Mémoires RAM, mémoire persistante
- Impact sur les performances

Vocabulaire:

Une “donnée” désigne une unité physique d’information : **une ligne d’une table**, codée sous forme d’un **enregistrement** (record)

Problématique de stockage

Un SGBD gère deux types de mémoire

- La **mémoire centrale**, volatile, rapide mais risque de perte
- La **mémoire secondaire**, persistante, stable mais lente

Les données sont toujours en mémoire secondaire dans des fichiers, et partiellement copiées en mémoire centrale

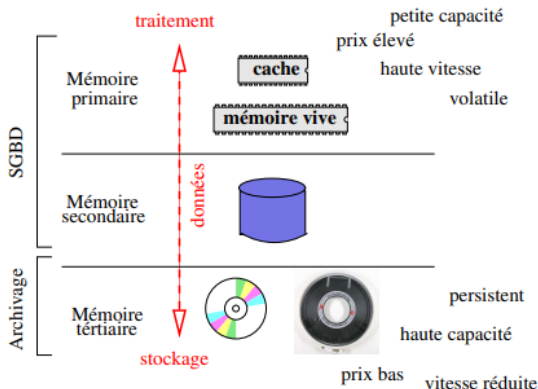
Une donnée, pour être traitée, doit **impérativement** être en **mémoire centrale**

- Les performances des SGBD sont mesurées essentiellement en nombre d'accès disque
- Comment gérer au mieux ces deux ressources ?



Les mémoires d'un ordinateur

- Les mémoires dans un ordinateur forment une hiérarchie
- Plus une mémoire est rapide, plus elle est chère, et moins elle est volumineuse



Critères essentiels :

- **Temps d'accès** : connaissant l'adresse d'une donnée, quel est le temps nécessaire pour aller la chercher à l'emplacement mémoire indiqué par cette adresse ?
 - ↪ Critère essentiel pour les opérations dites d'**accès direct**
- **Débit** : volume de données lues par unité de temps dans le meilleur des cas
 - ↪ Critère essentiel pour les opérations dites d' **accès séquentiel**

Important :

Tout accès aux données s'effectue avec l'une de ces deux opérations

Quelques ordres de grandeur

Rappel : 1 Go = 1 000 Mo et 1 To = 1 000 Go

Mémoire	Taille	Temps d'accès	Débit
Principale (RAM)	Qq Go	$\approx$ nanosec.	Qq Go/s
Disque	Qq To	$\approx$ 10 millisc.	100 Mo/s

- Un accès direct disque est (environ) un million de fois plus coûteux qu'en mémoire principale !
- Le débit d'un disque est 20, 30, 40 fois plus lent que le débit RAM

Importance pour les bases de données

- Un SGBD doit ranger sur disque les données ; parce qu'elle sont
 - trop volumineuses
 - persistantes (et doivent survivre à un arrêt du système)
- Le SGBD doit toujours amener les données en mémoire pour les traiter
- Si possible, les données utiles devraient résider le plus possible en mémoire
- Lire un enregistrement à partir d'une page/bloc de données sur disque dure beaucoup **plus longtemps** que de calculer un enregistrement à partir de données déjà en mémoire

Principe général :

la capacité à placer les données utiles au plus près du processeur (donc en RAM si possible) est un facteur essentiel de performance

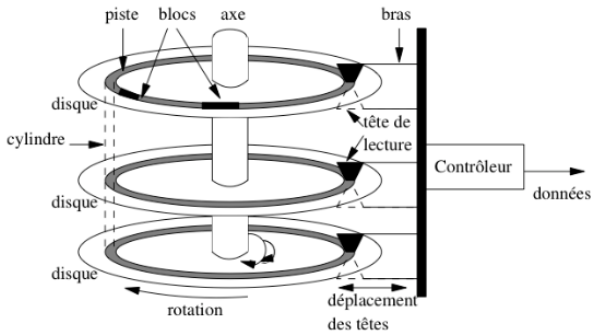
Architecture/Structure d'un disque

- Disque : surface magnétique, stockant des 0 / 1, divisé en secteurs
- Dispositif : les surfaces sont entraînées dans un mouvement de rotation ; les têtes de lecture se déplacent dans un plan fixe

Bloc/Page: ensemble de secteurs

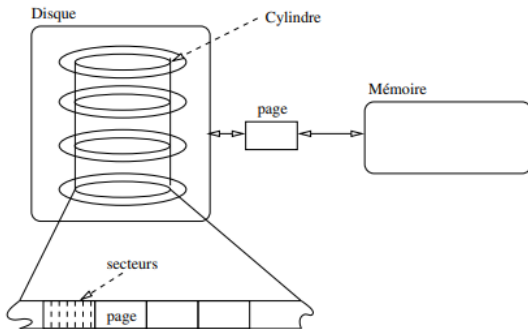
Piste: ensemble des blocs d'une surface lus au cours d'une rotation

Cylindre: ensemble de pistes situées sous les têtes de lecture



Architecture/Structure d'un disque

- Disque divisé en blocs de taille égale (même nombre de secteurs contigus)
- Le **bloc/page** = l'unité d'échange entre les **mémoires secondaire** et **principale**
 - ↔ **Coût** des opérations = **nombre de lectures/écritures de blocs**
- **Adresse bloc**: $N^{\circ}[\text{disque} + \text{piste où se trouve le bloc} + \text{bloc sur la piste}]$
- Têtes de lecture synchrones sur chaque face d'un cylindre



Accès Disque = latence

Adresse bloc: $N^{\circ}[\text{disque} + \text{piste où se trouve le bloc} + \text{bloc sur la piste}]$

- **délai de positionnement** pour placer la tête sur la bonne piste
- **délai de latence** pour attendre que le bloc passe sous la tête de lecture
- **temps de transfert** pour attendre que le (ou les) bloc(s) soient lus et transférés

La latence d'un disque :

pas possible de lire une donnée avant qu'elle passe sous une tête de lecture
→ la latence (qq ms) est un facteur pénalisant

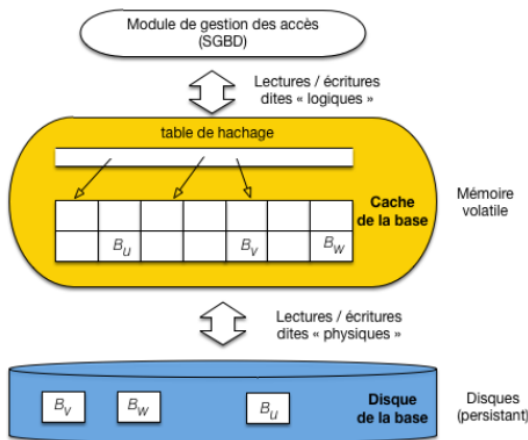
Granularité d'entrée/sortie : le bloc

- Une **entrée-sortie (I/O)** $\Rightarrow$ **Page** disque = **Bloc**
 - **Bloc**
 - $\approx$ **Secteur** sur le disque
 - = 8 Ko (par défaut sous Oracle)
- Un fichier de données : plusieurs pages/blocs disques
- Plusieurs tuples/enregistrements par page/bloc (dépend de la taille d'un tuple)

Adresse = numéro du disque ; de la piste où se trouve le bloc ; du numéro du bloc sur la piste

Gestion des mémoires: Le cache et le disque

Paramétrage des SGBD : partie de la **mémoire centrale** qui sert de **cache** (Mémoire Centrale, MC, RAM)



- Toute opération de lecture contient l'**adresse du bloc** à lire
 - Si la **donnée fait partie d'un bloc dans le cache**, le SGBD prend le bloc, accède à la donnée dans le bloc, et la retourne (**lecture logique**)
 - Sinon il faut d'abord **lire un bloc du disque**, et le **placer dans le cache**, pour se ramener au cas précédent (**lecture physique**)
- Hypothèse de localité : le SGBD garde en mémoire les blocs après utilisation, afin d'exploiter à la fois
 - la localité spatiale : les autres données du bloc
 - la localité temporelle : il est probable qu'on va relire la donnée dans peu de temps

Mesure l'efficacité d'une mémoire cache :

$$\text{hit ratio} = \frac{\text{nb de lectures logiques} - \text{nb lectures physiques}}{\text{nb de lectures logiques}}$$

- ⇒ Si toutes les lectures logiques (demande de bloc) aboutissent à une lecture physique (accès au disque) : *hit ratio* = 0
- ⇒ Aucune lecture physique (tout est en mémoire) : *hit ratio* = 1

Un bon Hit ratio ?

- Allouer le plus de mémoire possible au SGBD
- Limiter la taille de la base (attention aux longs textes, aux images, ...)
- surtout : il faut que les données utiles soient en mémoire centrale
- Intuition : certaines parties de la base sont beaucoup plus lues que d'autres : il faut qu'elles tiennent en mémoire

Que faire quand la mémoire est pleine ?

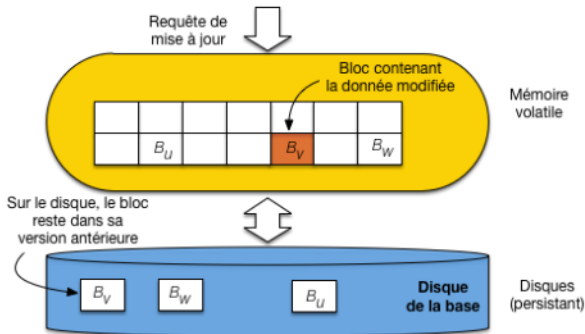
⇒ On applique le principe de localité : l'algorithme le plus courant est dit **Least Recently Used (LRU)**

- La 'victime' est le bloc dont la **dernière date de lecture logique** est la plus **ancienne**
- Ce bloc est alors soustrait de la mémoire centrale
- Le nouveau bloc vient le remplacer

Conséquence : le contenu du cache est une image fidèle de l'activité récente sur la base de données

Opération de mise à jour : Approche naïve

- 1 Trouver le bloc (comme pour une lecture), modifier la donnée
- 2 Ecrire le bloc de suite ou non ? Deux mauvais choix (pourquoi ?)

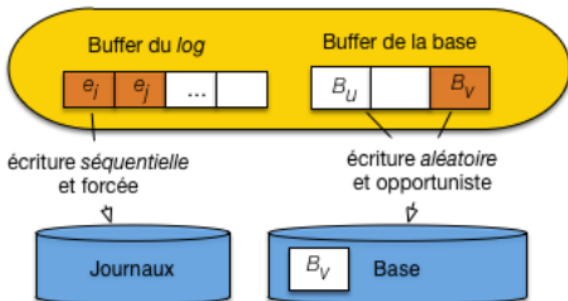


- ↪ **garder le bloc modifié en mémoire**, en attendant qu'une opportunité se présente pour effectuer l'écriture sur le disque; dans l'intervalle, toute **panne** du serveur entraîne la **perte** de la modification;
- ↪ **écrire le bloc sur le disque** pour remplacer la version non modifiée, et on se ramène à des **écritures non ordonnées, aléatoires**¹ et donc **pénalisantes**

¹accès aléatoire = accès direct = par adresse $\neq$ accès séquentiel

La bonne méthode : le fichier journal

- 1 Gestion de
 - ↪ la base (accès aléatoire (par adresse))
 - ↪ un fichier séquentiel, le journal (log)
- 2 au moment de l'écriture effective, l'opération sera probablement plus "rentable" qu'avec la solution naïve



- ↪ le bloc modifié n'est pas écrit sur disque
- ↪ la commande de mise à jour est écrite séquentiellement dans le log
- ↪ le bloc modifié est écrit sur disque de manière opportuniste (cache plein, serveur arrêté, ...)

Le principe de localité et ses conséquences

- L'**ensemble des données** utilisées par une application pendant une période donnée forme souvent un **groupe** bien identifié
 - **Localité spatiale** : si une donnée d est utilisée, les données proches de d ont de fortes chances de l'être également
 - **Localité temporelle** : quand une application accède à une donnée d , il y a de fortes chances qu'elle y accède à nouveau peu de temps après
 - **Localité de référence** : si une donnée $d1$ référence une donnée $d2$, l'accès à $d1$ entraîne souvent l'accès à $d2$

Le SGBD cherche à **optimiser le placement** les données "**proches**" de celles en cours d'utilisation en déplaçant dans la hiérarchie des mémoires des **groupes de données proches** de la **donnée utilisée à un instant t**

La représentation physique d'une base de données (par le SGBD)

- les enregistrements et leur adressage
- le stockage des enregistrements dans des blocs
- la séquence des blocs formant un fichier

Une application n'accède jamais directement à la représentation physique

- Les données sur disque sont stockées dans des fichiers
 - Un **fichier** occupe plusieurs **secteurs/blocs** sur disque
 - L'accès aux fichiers est géré par le système de gestion de fichiers du système d'exploitation
 - Un fichier est identifié par son nom
- Un **fichier** stocke un **ensemble d'enregistrements**
 - Un **enregistrement**/article est une **séquence de champs** (attributs)
- Deux types d'enregistrements
 - **enregistrements en format fixe** : la taille de chaque champ est fixée
 - **enregistrements en format variable** : la taille de certains champs est variable

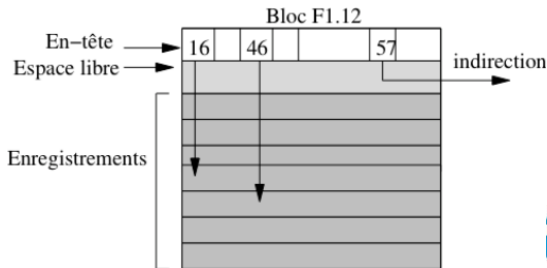
Enregistrements et blocs

- Les **enregistrements** sont stockés dans des **blocs**
 - En général $\text{taille enregistrement} < \text{taille bloc}$
 - Adresse enregistrement (**ROWID**) = (adresse bloc, indice de l'enregistrement dans le bloc)
 - Important: ne pas modifier l'adresse d'un enregistrement si celui-ci est référencé!
- **Organisation des blocs**
 - Nombreuses variantes: en fonction du type d'enregistrement (taille fixe/variable), de l'organisation du fichier (séquentielle, triée, ...)
 - **Bloc = entête de bloc + zone enregistrements**
 - La **zone d'enregistrements**: divisée en **cellules**
 - De taille constante (enregistrements de taille fixe)
 - De taille variable (enregistrements de taille variable)
 - **Adresse d'un enregistrement dans le bloc** = N° d'ordre de la cellule occupée
 - Suppression d'un enregistrement $\Rightarrow$ sa cellule devient disponible pour accueillir ultérieurement un autre enregistrement



Organisation générale d'un bloc

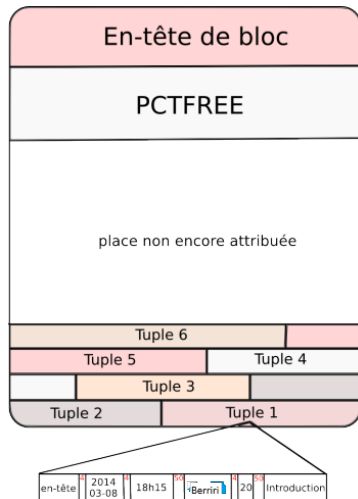
- Entête: répertoire de cellules + infos de chaînage des blocs
 - Répertoire de cellules: nb. de cellules allouées + tableau de descripteurs de cellule
 - Descripteur de cellule: adresse dans le bloc + indicateur d'occupation
- **Zone enregistrements = zone cellules + zone libre**
 - Zone cellules: cellules occupées + cellules libres
 - Suppression enregistrement: cellule marquée libre dans le répertoire
 - Ajout enregistrement: on cherche une cellule libre de taille suffisante, sinon on alloue une nouvelle cellule dans la zone libre



Page dique/ Bloc (Oracle)

Une page/bloc^a contient :

- Entête^b:
 - Type de données (relation, index ...)
 - Schéma de données (attributs, ...)
 - Pointeurs sur places vides, page suivante et précédente
- Place vide pour mises à jour
 - PCTFREE^c / PCTUSED
- Les données
 - Enregistrements en entier
 - Entête de tuple^d
 - À chaque attribut s'associe sa taille



^aPar défaut 8192 o

^bEntre 100 et 200 o

^cPar défaut 10%

^dVariable

Enregistrements, Types de données

- Un enregistrement = une suite de champs stockant les valeurs des attributs
- Certains champs ont une taille variable ; d'autres peuvent être à NULL : pas de valeur

Types numériques

Type	Taille	Interval de valeurs
TINYINT	1 octet	0 à 255 ou -128 à 127
SMALLINT	2 octets	-32768 à 32767 ou 0 à 65535
MEDIUMINT	3 octets	-8388608 à 8388607 ou 0 à 16777215
INT, INTEGER	4 octets	-2147483648 à 2147483647 ou 0 à 4294967295
BIGINT	8 octets	$\sim 10^{19}$
FLOAT(p)	4 octets if $0 \leq p \leq 24$, 8 octets if $25 \leq p \leq 53$	nombre de décimales
FLOAT	8 octets	Par défaut, 53 décimales
DOUBLE(p), REAL	8 octets	
DECIMAL(M,D), NUM(M,D)	Variable	
BIT(M)	$\sim (M + 7) / 8$ octets	

Types Dates et Heures

Type	Taille
DATE	3 octets
TIME	3 octets
DATETIME	8 octets
TIMESTAMP	4 octets
YEAR	1 octet

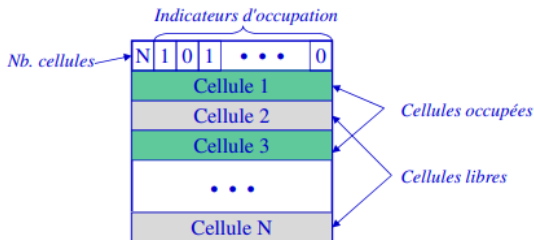
Types textuel

Type	Taille
CHAR(M)	M octets ou M x 2 octets, $0 \leq M \leq 255$
BINARY(M)	M octets, $0 \leq M \leq 255$
VARCHAR(M)	M + 1o (0-255 o), M + 2o (> 255 o)
BLOB, TEXT	L + 2 octets, where $L \leq 2^{15}$
ENUM('v1','v2',...)	1o (255 valeurs), 2o (65 535 valeurs)
SET('v1','v2',...)	1, 2, 3, 4, or 8 octets (64 valeurs maximum)

Enregistrements à taille fixe

- Plus simple à gérer
 - Pas besoin de garder dans le répertoire la position de chaque cellule
 - ↳ On garde seulement un bit d'occupation par cellule
 - L'accès aux champs d'un enregistrement est simple
 - ↳ La position de chaque champ est toujours la même
 - Pas de problèmes de débordement d'un enregistrement ou d'un bloc

⇒ Economie de place dans le bloc et accès plus rapide aux enregistrements/champs



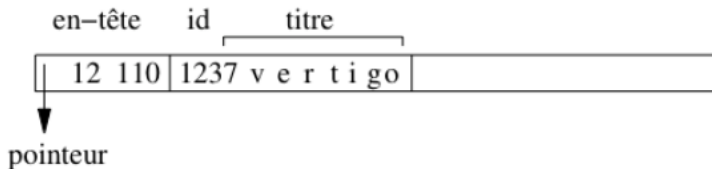
Enregistrements à taille variable et valeurs *NULL*

- En pratique : certains champs ont une taille variable ou sont à *NULL*
 - ↪ champs de taille variable : on précède la valeur par la taille exacte
 - ↪ valeurs *NULL* : on peut indiquer une taille 0 (Oracle) ; on peut créer un “masque” de bits ...
- La norme n'impose pas de choix, mais dans tous les cas il faut gérer une information complémentaire sur les enregistrements, et la placer dans un entête

En-tête d'enregistrement

Les informations complémentaires sont stockées dans l'entête d'un enregistrement

- table Film (id:INT, titre:VARCHAR(50), année:INT)
- Enregistrement (123, 'Vertigo', NULL)
 - *id* : stocké sur 4 octets,
 - *titre* : 8 octets, dont 1 pour la longueur
 - L'entête de l'enregistrement contient
 - ↳ un pointeur vers le schéma de la table
 - ↳ sa longueur totale (soit $4 + 8 = 12$)
 - ↳ un masque de bits 110 indiquant que le 3ème champ est à NULL



Stockage des enregistrements dans un fichier

Le **nombre maximal d'enregistrements**, de taille E chacun, pour un **bloc** de taille B est donné par ²

$$\lfloor B/E \rfloor$$

Ex. **fichier stockant une table sans attributs de longueur variable** (pas les types VARCHAR ou BIT VARYING)

- un enregistrement a une taille fixe obtenue en effectuant la somme des tailles de chaque attribut. Supposons que cette taille = 84 octets
- il y a 300000 enregistrements dans la table
- la taille du bloc = 4096 octets
- chaque bloc contient un entête de 100 octets

$$\hookrightarrow \text{nbre enregistrements dans un bloc} = \lfloor \frac{4096-100}{84} \rfloor = 47$$

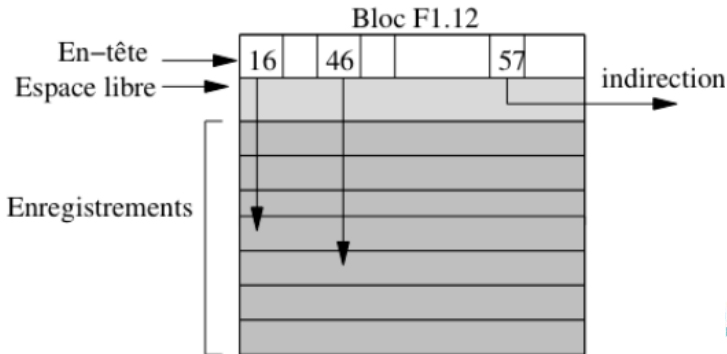
$$\hookrightarrow \text{taille du fichier}^3 = \lceil \frac{\text{nbre total enregistrements}}{\text{nbre enregistrements dans un bloc}} \rceil = \lceil \frac{300000}{47} \rceil = 6383 \text{ blocs}$$

² $\lfloor x \rfloor$ désigne le plus grand entier inférieur à x

³ $\lceil x \rceil$ désigne le plus petit entier supérieur à x

Blocs/Enregistrements

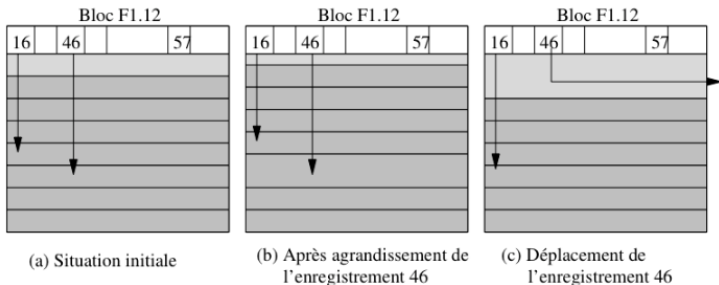
- On veut qu'un enregistrement soit stocké de manière stable pour avoir une adresse fixe ; Mais
 - ↳ le cas où la taille d'un enregistrement varie
 - ↳ une méthode pour gérer un déplacement si nécessaire
- Une solution (Oracle) : chaque bloc gère les adresses internes de ses enregistrements



- Avec le schéma précédent, l'adresse d'un enregistrement est constituée
 - ↪ D'une adresse physique, celle du bloc. Ex : F1.12
 - ↪ D'une adresse logique, interne au bloc. Ex : 16
- L'adresse est fixe et utilisée pour **les accès directs** (indexation)
- Avec cette solution :
 - ↪ un enregistrement peut changer de taille, avec **réorganisation** interne
 - ↪ Plus de place dans le bloc ? On peut déplacer l'enregistrement

Réorganisation du stockage

- Un enregistrement s'agrandit
 - reste de la place dans le bloc:
 - Modifier l'organisation interne ; la table locale d'adressage est modifiée
 - plus de place dans le bloc :
 - Déplacer l'enregistrement et créer un chaînage dans l'en-tête du bloc
 - Le chaînage pénalise les performances ⇒ si possible laisser de l'espace libre dans un bloc
- Ex. On agrandit deux fois successivement l'enregistrement *F1.12.46*



Fichier = séquence de blocs

- Un fichier est simplement une séquence de blocs contenant les données d'un même objet (table, index)
- La qualité d'un fichier est conditionnée par
 - ↪ la bonne utilisation de l'espace (idéalement tous les blocs sont pleins)
 - ↪ le stockage le plus contigu possible (même piste, même cylindre, etc)
 - ↪ peu ou pas de chaînage
- Si les blocs sont très dispersés sur le disque, on aboutit à une fragmentation très pénalisante

- Essentiellement :
 - En l'absence d'index, le seul moyen de rechercher un enregistrement est de parcourir séquentiellement le fichier
 - Si le fichier est trié il est possible d'effectuer des recherches par dichotomie
- En général, les fichiers gérés par les SGBD sont des fichiers séquentiels, sans ordre

Exemple de stockage d'une table *Cours*

Bloc	CODE	Intitule	Responsable	nb_auditeur
B1	glg102	SGBD Aspects Système	Asma Berriri	50
	glg203	Complément Objet	Frederic Voisin	100
	nfa114	Projets	Anastasia Bezerianos	30
B2	smb114	Compilation	Frederic Voisin	40
	nfe212	Intro base de données	Asma Berriri	45
	nfe006	Programmation C++	Emmanuelle Frenoux	42
B3	nfa003	Interfaces HM	Anastasia Bezerianos	37
	nfe115	Concepts Objet et Java	Aurélien Max	38
	nsy114	Vérif. and validation	Burkhart Wolff	72

- *CODE* et *nb_auditeur* codés sur 2 octets chacun, *Intitule* et *Responsable* sur 60 o chacun
- entête d'un bloc de données de 18 octets
- notre SGBD constitue des blocs (pages) de 512 octets
- Quel est la taille en nombre de blocs de la table *Cours* en supposant qu'un enregistrement ne peut pas chevaucher deux blocs?

↪ Taille d'un enregistrement $(2 + 2 + 60 + 60) = 124$ octets

↪ Entête bloc fait 18 o, un bloc fait 512 o. bloc utile : $512 - 18 = 494$ o

↪ Nb enregistrements par bloc : $\lfloor \frac{494}{124} \rfloor = 3$ (on ne coupe pas d'enregistrement en 2)

↪ Il y a 9 enregistrements, donc il faudra $\lceil \frac{\text{nbre total enregistrements}}{\text{nbre enregistrements dans un bloc}} \rceil = \lceil \frac{9}{3} \rceil = 3$ blocs



Exemple simple d'accès

```
SELECT Responsable FROM Cours WHERE Intitule='Concepts Objet et Java'
```

$$\pi_{\text{Responsable}}(\sigma_{\text{Intitule}='Concepts\ Objet\ et\ Java'}(\text{Cours}))$$

- Si **pas d'index sur l'attribut *Intitule*** :
 - Parcourir séquentiellement chaque **page/bloc** (physique) de la relation *Cours* : **FULL ACCESS**
 - Si l'enregistrement contient 'Concepts Objet et Java', il est projeté
 - **Coût de l'opération** : Nombre de blocs de *Cours* = **3 lectures**

- La gestion des fichiers est entièrement à la charge du SGBD
 - Les enregistrements sont stockés dans des blocs/pages
 - Le contenu des blocs s'adapte au changement de taille d'un enregistrement après insertion
 - Il faut un dispositif stable d'adressage de chaque enregistrement
 - Les fichiers sont des séquences de blocs, les moins fragmentés possibles
- L'adressage stable est essentiel pour permettre l'indexation et l'accès direct (par adresse) aux enregistrements